

AeSlides: Incentivizing Aesthetic Layout in LLM-Based Slide Generation via Verifiable Rewards

Verified reference summary • Year: 2026 • Source bibliography: docs/REFERENCES.md

What the paper contributes

AeSlides studies how to make LLM slide generation optimize for visual quality through verifiable reward signals. It treats aesthetics as observable layout defects and rewards rather than as purely subjective prose feedback. The paper is especially relevant because it makes slide quality operational: aspect ratio, excessive whitespace, element collision, and visual imbalance become measurable conditions.

Method / evaluation stance

The implementation path is render-first: metrics are evaluated over browser-rendered slide output. That makes the measurements close to what a viewer sees, but it also couples the evaluator to Playwright, screenshots, fonts, viewport choices, and proprietary or paper-specific formulas.

How slidekit uses it

For slidekit, the important transfer is not the rendering stack; it is the premise that layout quality can be checked automatically. slidekit ports the same categories into native geometry: whitespace becomes content coverage and empty-region analysis; collision becomes exact rectangle intersection; balance becomes area-weighted centroid distance from the slide center.

YAML vs HTML takeaway

This paper is a strong argument for verifiable rewards, but slidekit deliberately chooses a source-first implementation. YAML plus ResolvedDeck geometry gives deterministic, auditable checks before emission. HTML remains useful when evaluating final pixels or training reward models, but the production deck source should stay structured.

Bottom line

The paper supports a pragmatic split: use research-style rendered artifacts when the question depends on final pixels or human preference collection, but keep slidekit production decks in a constrained YAML IR so layout, linting, and agent fixes remain deterministic, inspectable, and cheap.

Primary slidekit mapping	balance / alignment / whitespace / overlap / contrast / richness / consistency, depending on the paper
Operational risk	HTML metrics can drift with rendering environment; YAML metrics can miss pixel-only effects. Use each where it is strongest.